*** Calling the no argument constructor of the current class :**

```
package com.ravi.super_keyword;

class Parent
{
    public Parent()
    {
        IO.println("No Argument constructor of Parent class");
    }

    public Parent(String name)
    {
        IO.println("Parameterized constructor "+name);
    }
}

class Child extends Parent
{
    public Child()
    {
        //Therefore we can either call no argument Constructor OR parameterized constructor
        super("Ravi");
        IO.println("No Argument constructor of Child class");
    }
}

public class CallingTheNoArgumentReturnThisClass
{
    public static void main(String[] args)
    {
        new Child();
    }

}
```

Note : From the above program, it is clear that from Child class constructor, We can either call no argument constructor OR parameterized constructor of super class.We cannot call no argument constructor of Child class constructor.

* In order to call current class constructor, we should use this() OR this("Java")

```
package com.ravi.super_keyword;

class Parent
{
    public Parent()
    {
        IO.println("No Argument constructor of Parent class");
    }

    public Parent(String name)
    {
        this()
        IO.println("Parameterized constructor "+name);
    }
}

class Child extends Parent
{
    public Child()
    {
        super("Java");
        IO.println("No Argument constructor of Child class");
    }

    public class CallingTheNoArgumentReturnThisClass
    {
        public static void main(String[] args)
        {
            new Child();
        }

    }
}
```

Hierarchical Inheritance Program :

```
package com.ravi.super_keyword;

class Shape
{
    protected int x;

    public Shape(int x)
    {
        this.x = x;
        IO.println("x value is :"+x);
    }
}
class Square extends Shape
{
    public Square(int side)
    {
        super(side);
    }

    public double getAreaOfSquare()
    {
        double area = super.x * this.x;
        return area;
    }
}

class Rectangle extends Shape
{
    protected int breadth;

    public Rectangle(int length, int breadth)
    {
        super(length);
        this.breadth = breadth;
    }

    public double getAreaOfRectangle()
    {
        double area = this.x * this.breadth;
        return area;
    }
}

public class SuperDemo
{
    public static void main(String[] args)
    {
        int side = Integer.parseInt(IO.readln("Enter the value of side :"));
        Square s = new Square(side);
        IO.println("Area of the Square is : "+s.getAreaOfSquare());

        int length = Integer.parseInt(IO.readln("Enter the value of length :"));
        int breadth = Integer.parseInt(IO.readln("Enter the value of breadth :"));

        Rectangle rectangle = new Rectangle(length, breadth);
        IO.println("Area of Rectangle is : "+rectangle.getAreaOfRectangle());

    }

}
```

```
class A
{
    public String message;
    {
        this(false);//call this constructor
    }
}
class B extends A
{
    public B()
    {
        super(getMessage());//before calling super class constructor we cannot call non static method
    }

    public String getMessage()
    {
        return "Java";
    }
}

public class InheritanceTest
{
    public static void main(String[] args)
    {
        B b = new B();
    }
}
```

```
class A   //ONE IS TO
{
    {
        IO.println("1");
    }

    static
    {
        IO.println("2");
    }

    {
        IO.println("3");
    }
}
class B extends A
{
    {
        IO.println("4");
    }

    static
    {
        IO.println("5");
    }

    {
        IO.println("6");
    }
}

public class InheritanceTest
{
    public static void main(String[] args)
    {
        new B();
    }
}
```

```
package com.ravi.super_keyword;

class X
{
    protected int x = 100;
}
class A extends X
{
    protected int x = 200;

    public static void access()
    {
        X X = new X();
        IO.println(X.x);

        //If the super-class type, we can assign the object of sub-class
        X = new A();
        IO.println(X.x);
    }
}
public class TS
{
    public static void main(String[] args)
    {
        A.access();
    }
}
```

PHP Is achieve single level inheritance :

```
package com.ravi.single_level_inheritance;

class EMP
{
    protected int id;
    protected String name;
    protected double salary;

    public EMP(int id, String name, double salary)
    {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

}

class PMP extends EMP
{
    protected String department;
    protected String designation;

    public PMP(int id, String name, double salary, String department, String designation) {
        super(id, name, salary);
        this.department = department;
        this.designation = designation;
    }

    @Override
    public String toString() {
        return "EMP [id=" + id + ", name=" + name + ", salary=" + salary + ", department=" + department
                + ", designation=" + designation + "]";
    }

}

public class EmployeeDemo {

    public static void main(String[] args)
    {
        PMP p = new PMP(1, "Ravi", 4000, "Civil", "Doctor");
        IO.println(p);
    }

}
```